Failure and resilience in Pastry

COP6539 Project 3, bonus. Ahmed Rageeb Ahsan, Saiful Islam

In the main project nothing ever fails. For the bonus we added the three failures from the assignment, implemented the recovery methods described in the Pastry paper, and measured which method helps against which failure. All experiments use 1,000 nodes that each make three lookups. Every point is the average of three runs.

How we tested resilience

Failure models. All three start after the network is built and the objects are stored, so we are measuring whether lookups still work while things fail. Each one is controlled by one parameter.

- `kill=p` : **a node dies.** A fraction *p* of the peers is killed with no warning when the lookups start. Any message on its way to them is lost.
- `drop=q` : **a connection dies temporarily.** Each message is lost with probability *q*, so the same link can work on the next try.
- `links=r` : **a connection dies permanently.** A fraction *r* of node pairs can never reach each other. We decide this by hashing the two node ids, so all nodes agree on it without sharing anything, and a dead link stays dead on every retry.

Defences. These follow section 2.5 of the Pastry paper, plus replication from PAST. Each one can be turned on separately so we can see what it adds.

- **Failure detection:** each node monitors the nodes in its leaf set and routing table. We use Erlang monitors for this instead of the keep-alive messages in the paper.
- **Repair:** when a leaf set node fails, ask the farthest live leaf on that side for its leaf set. When a routing table entry fails, ask another node in the same row for its entry. Both come from the paper.
- **Replication:** each object is stored on the four nodes closest to its key, like PAST does, so it survives if one of them dies.
- **Retry:** each lookup has a timeout. If it times out, or reaches a node that doesn't have the object, it is retried up to three times in total, either on the same route or on a random one.

Each request is counted exactly once, as found, not found, or failed after its last timeout. If a late reply comes back for an attempt we already retried, we ignore it, so nothing is counted twice. We also checked the random routing against the brute force closest node, and it always reached the right one.

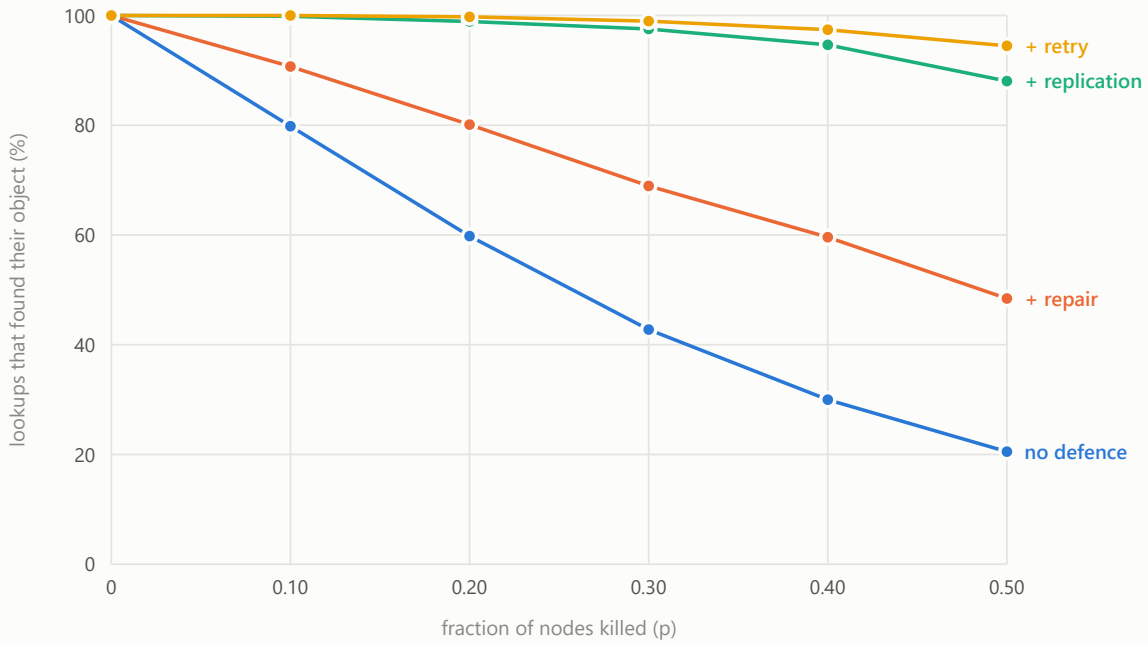
---

1. A node dies

no defence + repair + replication + retry

Node death

Each line includes the defences of the lines before it.



Repair fixes the routing but can't bring back lost data. At p = 0.2, 59.8% of lookups succeed with no defence, and repair brings this up to 80.1%. The loss that remains is about the same as the fraction of nodes killed, at every value of p (see the table below). These are objects whose only copy was on a node that died. After repair the lookup goes to the right node, but the object is not there, and better routing can't fix that.

Replication is what keeps the data. With it, p = 0.2 goes up to 98.9%, and it is still 88.0% when half of the nodes die. Adding retry helps more than we expected, up to 94.5% at p = 0.5. It catches lookups that were sent in the short time before repair finished.

With all defences on, we get close to the best possible result. An object with four copies is only lost if all four nodes die, which happens with probability p^4. The measured curve is very close to $1 - p^4$, so the lookups that still fail are for objects that could not have been saved. In the same way, repair alone follows $1 - p$, which is the chance that a single copy survives.

Nodes killed (p)	Repair only	$1 - p$	Every defence	$1 - p^4$
0.1	90.7%	90.0%	100.0%	99.99%
0.2	80.1%	80.0%	99.8%	99.84%
0.3	68.9%	70.0%	99.0%	99.19%
0.4	59.6%	60.0%	97.4%	97.44%
0.5	48.4%	50.0%	94.5%	93.75%

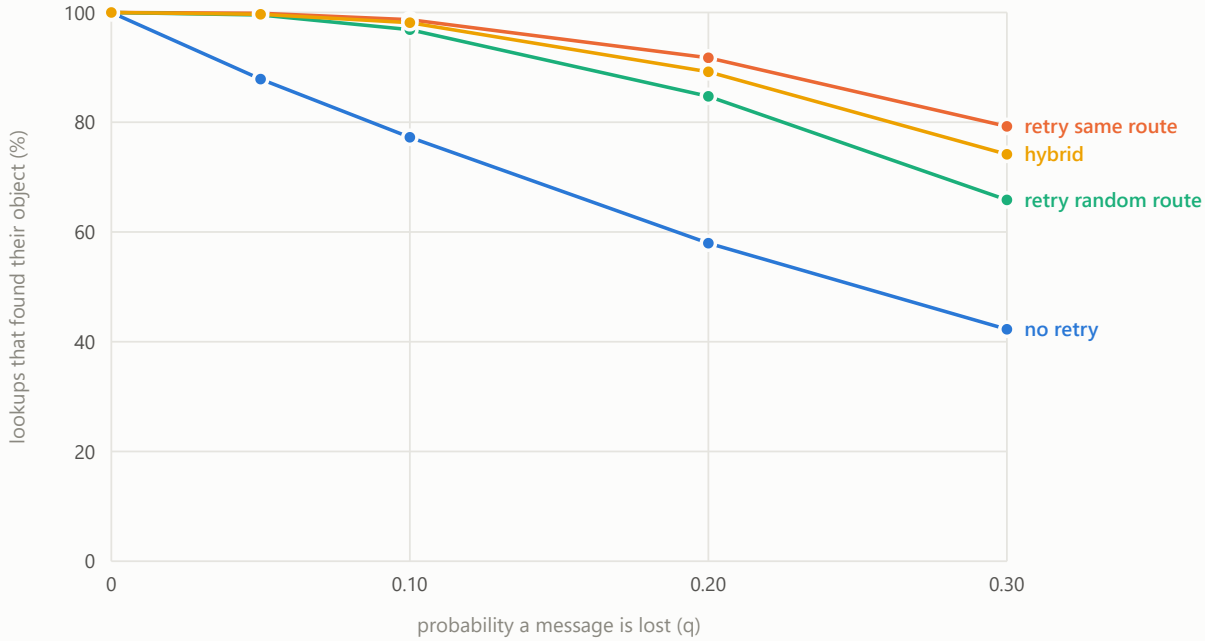
So repair and replication solve two different problems. Repair makes sure the message can get to the right node, and replication makes sure the object is still there. To handle node death we need both.

2. A connection dies temporarily

no retry retry same route retry random route hybrid

Temporary connection failure

Each message is lost with probability q.



A lookup takes about 2.48 hops, and if any of them is lost the lookup fails. So we expect it to succeed with probability $(1 - q)^{2.48}$. For q = 0.2 that gives 57.4%, and we measured 58.0%.

Retrying fixes most of this, and here **retrying on the same route works best**, for every value of q. At q = 0.2 it gets 91.7%, against 84.7% for a random route. Each try on the same route fails independently, so after three tries we expect $(1 - 0.574)^3$ of lookups to fail, which predicts 92.3%. A random route is a bit longer, and every extra hop is another chance to lose the message.

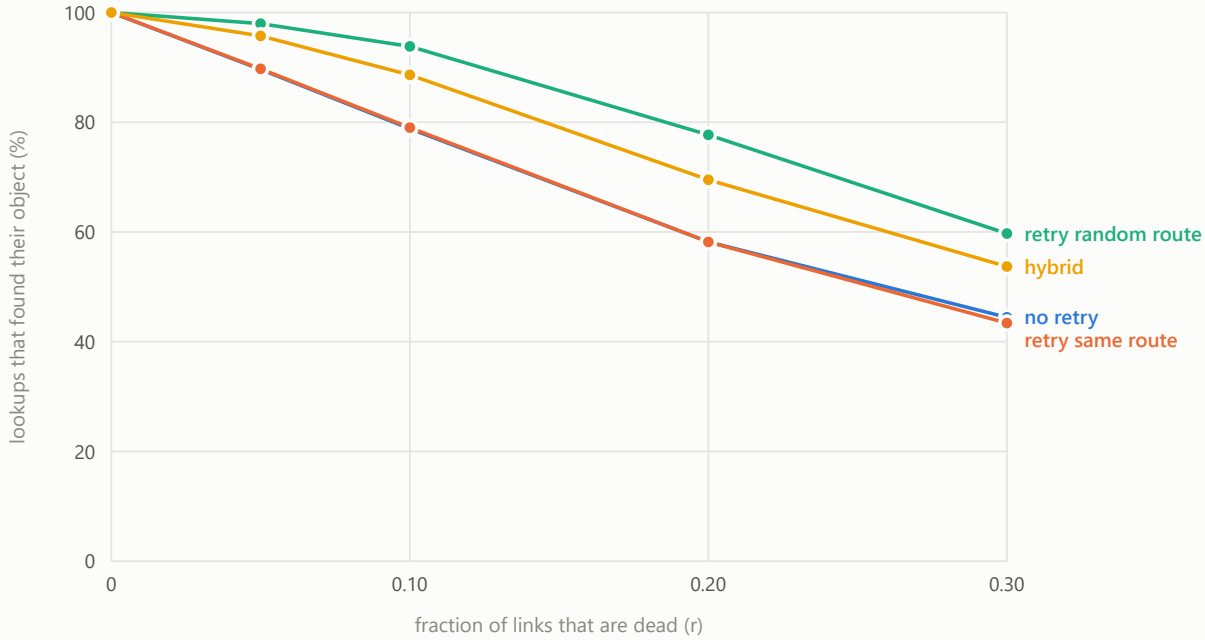
---

3. A connection dies permanently

no retry retry same route retry random route hybrid

Permanent connection failure

A fraction r of node pairs can never communicate.



Here it is the other way around. Retrying on the same route does not help at all: 58.2% at r = 0.2, compared to 58.2% with no retry. The two lines are on top of each other for every value of r. Pastry always picks the same next hop for a key, so a retry from the same node goes the same way and hits the same dead link again. Only a **random** route gets around it, and that gets 77.7%. This is why the Pastry paper suggests randomizing the route when retrying.

Conclusion

> **No retry policy is best for both kinds of connection failure, and a node can't tell which one it is dealing with.** Retrying on the same route is best for temporary failures but useless for permanent ones, and the random route is the opposite. When a lookup times out, the node doesn't know which kind of failure caused it.
>
> We also tried a mix, called *hybrid* in the charts: retry the same route once, then use random routes. It was never the best. It was always between the other two, at all eight failure levels we tested. At 20% it got 89.2% for temporary failures and 69.5% for permanent ones. With only three attempts, using one on the same route wastes it when the link is dead for good.
>
> Because of this the program uses random routes for retries by default. It has the best worst case: it loses a little on temporary failures but does much better on permanent ones, which is what we want when we don't know the type of failure.

Overall, each type of failure needed its own fix. Node death needed replication, temporary connection failures needed retry, and permanent connection failures needed retry with a *random* route.

How to reproduce

```
project3 1000 3 kill=0.2
project3 1000 3 kill=0.2 resilient
project3 1000 3 drop=0.2 attempts=3 retry=det
project3 1000 3 links=0.2 attempts=3 retry=rand
```